W4 High Performance Python (1)
List, Tuple, Dic., Namespace, and Generator

Department of Smart Mobility Engineering
Inwook, Shim
인하대학교
INHA UNIVERSITY

Previously

- Code Profiling
  - Introduction to Vanilla Julia set
  - print, @decorator, timeit, time, cProfile, snakeviz, line_profiler, py-spy
  - memory_profiler, mprof, memit, pdb,
  - dis

- Stack and Heap
- Some Key Issues
  - Trade-off
  - Performance variance
  - Bytecode

Today

- Goal
  - Right usage of List and Tuple
  - Time complexity for search algorithm

- Content
  - Big-O notation
  - Static and dynamic array
  - Linear search
  - Sorting and search
    - Insertion sort
    - Merge-sort
    - Binary search

Priori Knowledge for List and Tuple
- Big-O -

Python Array

- A collection of elements of the same type.
- Each element of the array is referenced by its index (memory address.)
- An array is a flat list of data with some intrinsic ordering.

- Python provided List and Tuple are the array structure.

  In array, the relative ordering of the element is important.

  *If we have a priori knowledge of the ordering, we can retrieve a data in the array in O(1).

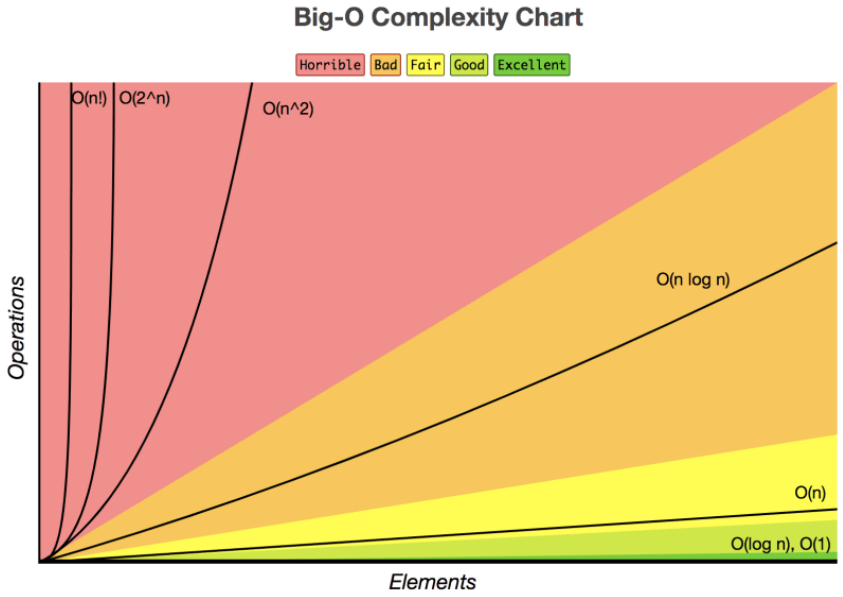
- Common mistake : Indexing an array outside of its boundary.

배열: 정해진 고유의 순서에 따라 데이터 나열
항목 순서가 데이터 그 자체만큼 중요

Big-O notation

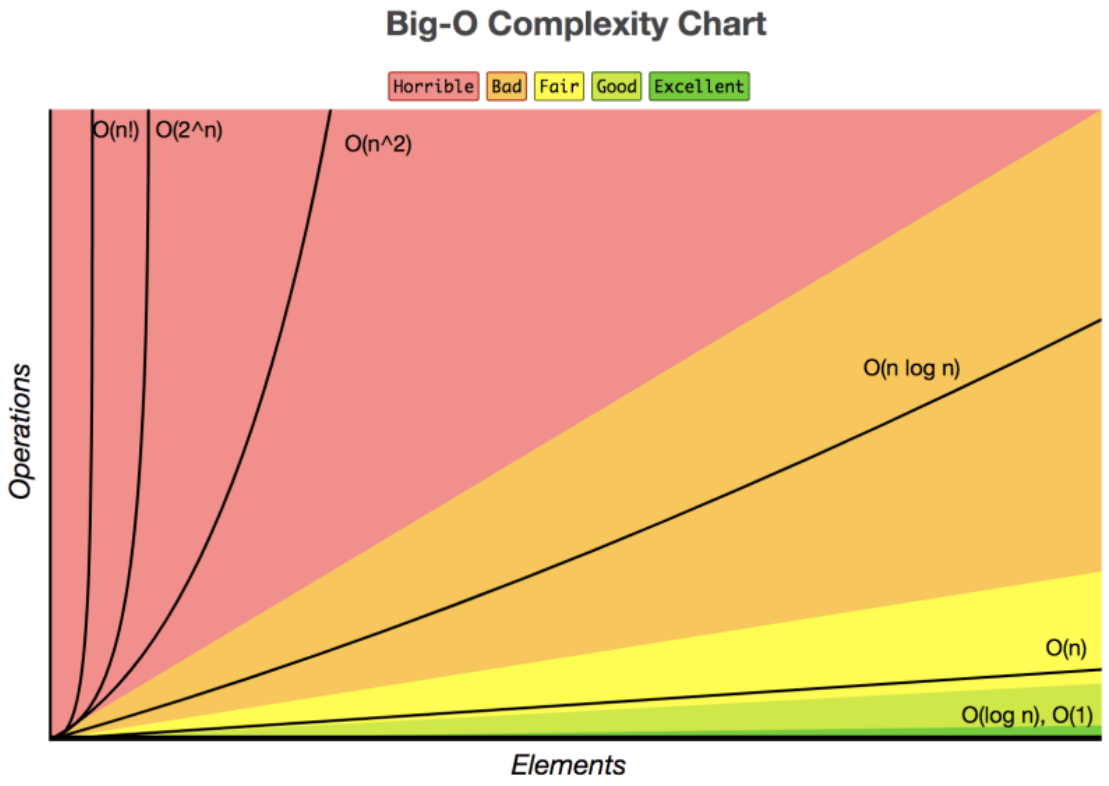
- Measurement unit to describe the performance or complexity of an algorithm.
  - Problem of Time Unit
    - Different computers have different processors and thus some computers will spend less time running an algorithm than others.
    - Some programming languages are faster than others.
  - Big-O notation is closer to standard. (Runtime complexity for worst-case scenario.)

    >> independent to computing power, code language



https://dev.to/sarah_chima/the-big-o-notation-an-introduction-34f7

Examples of Big-O notation



- O(1) – Constant Runtime

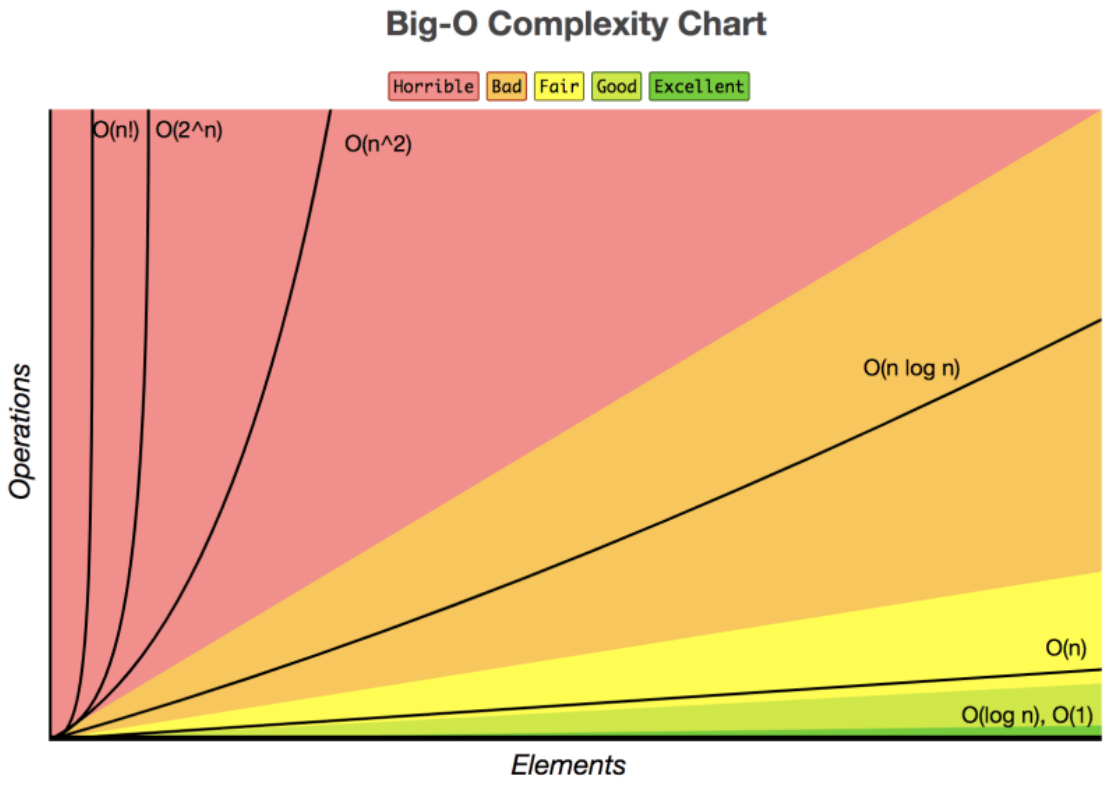
```python
def returnFirst(array):
  return array[0]
```

- O(n) – Linear Runtime

```python
def findIndex(array, value):
  for i in range(len(array)):
    if array[i] == value:
      return i
  return -1
```

- Question: What if the element is found before it reaches the last element in the array? In this case the runtime complexity change?

Examples of Big-O notation



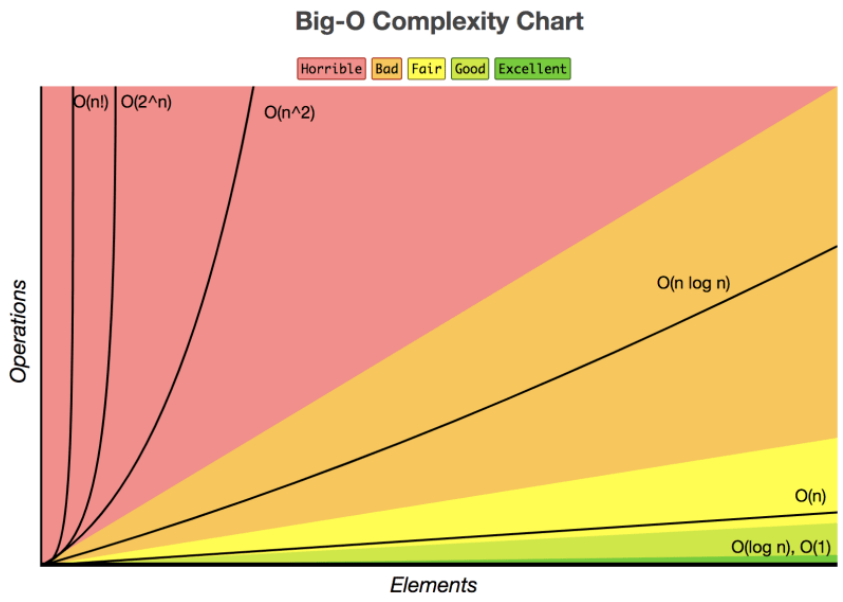
- $O(n^2)$ – Quadratic Runtime

```
def findIndex2D(array, value):
  for i in range(len(array)):
    for j in range(len(array[i])):
      if array[i][j] == value:
        return i, j
    #end for j
  #end for i
  return -1
```

*Deeper nested iteration will increase complexities.

- O(log n) – Linear Runtime
  - Binary search : reduce searching space.
- O(n log n) – Linearithmic Runtime
  - Most sorting algorithms

Examples of Big-O notation



- $O(2^n)$ – Exponential Runtime

```
def fibonacci(num):
  if num <= 1:
    return 1
  return (fibonacci(num - 2) + fibonacci(num - 1))
```

*Each increase of the data size produces doubled runtime.

- O(n!) – Factorial Runtime
  - Permutation

```
import itertools
arr = ['A', 'B', 'C']
nPr = itertools.permutations(arr, 3)
print(list(nPr))
```

>> [('A', 'B', 'C'), ('A', 'C', 'B'), ('B', 'A', 'C'), ('B', 'C', 'A'), ('C', 'A', 'B'), ('C', 'B', 'A')]

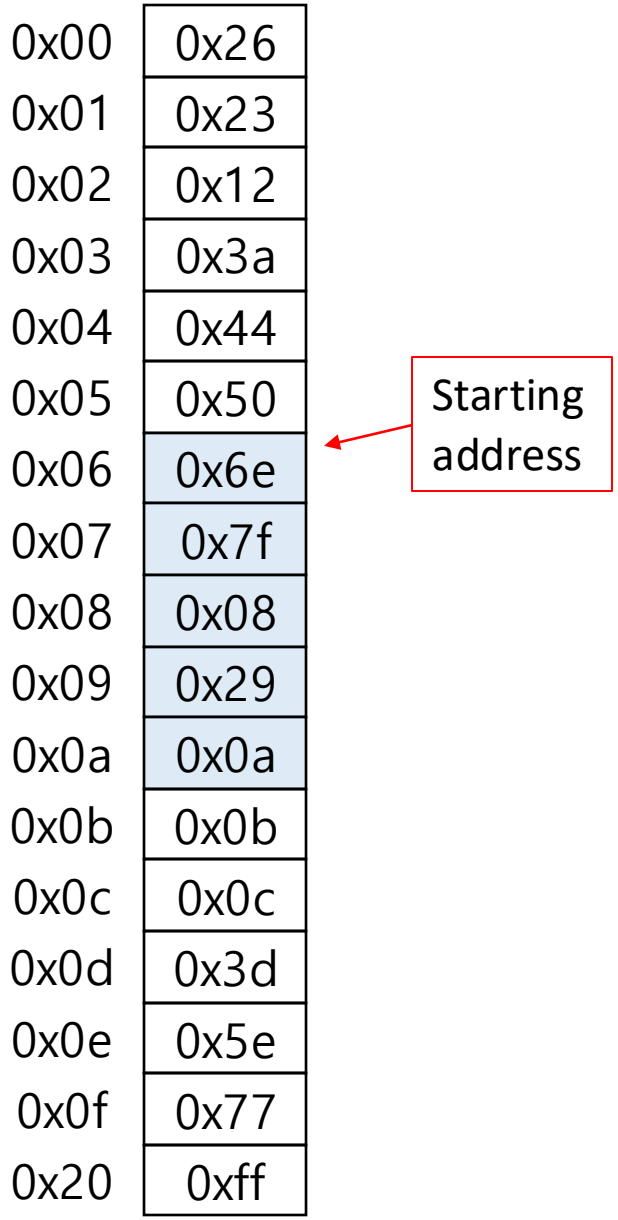
Lists and Tuples

- Lists are dynamic array
  - We can modify and resize the data. (e.g., sorting)
- Tuples are static array
  - The contents in Tuple are fixed and immutable.

- Python List stores data in a consecutive of buckets by reference.

  * The numbers in the buckets refer to the address data which actually stores a value.

Reference 기반으로 데이터를 저장하기 때문에, 모든 타입의 데이터를 저장할 수 있음. (*numpy는 정적으로 타입이 정해져 있음.)



Retrieve a specific elements from List

```
l = list(range(10))
%timeit l[5]
50.5 ns ± 1.01 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)

l = list(range(10_000_000))
%timeit l[500]
49.6 ns ± 0.614 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)

%timeit l[500_000]
49.6 ns ± 0.921 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)
```

- The data are stored in consecutive buckers. Therefore, if we want to retrieve element i^{th} from the array, we just read data position at starting address + i^{th}. O(1)

  * Name of array indicates starting address.

Priori Knowledge for List and Tuple

- Insertion-sort, Binary Tree, Merge-sort -

Unordered array

- Retrieve a specific element from unordered array.
- The most basic approach is <u>linear search</u>. O(n)

```
def findIndex(array, value):
  for i, item in enumerate(array):
    if item == value:
      return i
  return -1
```

list.index() use linear search →
Q: list.where() ??

```
l = [1,2,3,4,5,2,3]
l.index(2)
>> 1 (*return the index of the first matching value and exit)
```

- To increase searching speed, need to understanding of how the data is placed in memory, or re-arrangement of the bucket of the data *i.e.*, sorting. (Other approach: Hash-Table, use extra memory.)

Q: Find the index of 2012.
[2, 4, 6, 8, 10, 12, 30, 40, 102, 104, 106, 108, 2010, 2012, 2030, 2040]

*id est (that is)

Built-in sorting algorithm for Python List

1. Insertion sort
2. Binary Tree
3. Merge sort

- Sorting algorithm achieves better search performance.
- Intrinsic sorting algorithm for Python Lists is Tim-sort

```
arr = [5, 2, 8, 1, 4]
arr.sort()
```

- Tim-sort can sort a list in O(n) in the best case and O(n log n) in the worst case, which <u>heuristically combines insertion and merge-sort</u>.
  - **Insertion Sort**: Each iteration of the algorithm inserts the next element into the <u>current sorted part of the array</u>.
  - **Merge-Sort**: The Merge Sort algorithm is a sorting algorithm that is based on the <u>Divide and Conquer paradigm</u>. In this algorithm, the array is initially divided into <u>two equal halves</u> and then they are combined in a sorted manner.

We need to learn Binary Tree.

Binary Search Tree

- Once a list has been sorted, we can find our desired element using a binary search, which has an average case complexity of O(log n).

Binary search example

```
def binary_search(tree, value):
  ind_min, ind_max = 0, len(tree)
  while True:
    if ind_min > ind_max:
      return -1
    ind_root = (ind_min + ind_max) // 2

    if tree[ind_root] > value:
      ind_max = ind_root
    elif tree[ind_root] < value:
      ind_min = ind_root + 1
    else:
      return ind_root
```

>> Python std. library **bisect** module

Lists v.s. Tuples

- Pick the right data structure to build more efficient data analysis.

- Lists are dynamic array; they are mutable and allow for resizing.
- Tuples are static array; immutable, and the data cannot be changed after they are created.

  >> Tuple is a light-weighted data structure in comparison with List.

Tuple Example: Book Titles, author, year, …

List Example: the name of reader of a book,

튜플은 변치 않는 대상의 여러 속성을 보관하는데 유용
리스트는 서로 이질적인 객체들의 모음 (reference)

- Note that lists and tuples can hold different data types (because of storing the addresses.)

  >> overhead.

*이 오버해드는 numpy로 효율적으로 다룰 수 있음.

Lists

Lists

https://wiki.python.org/moin/TimeComplexity

- Data update

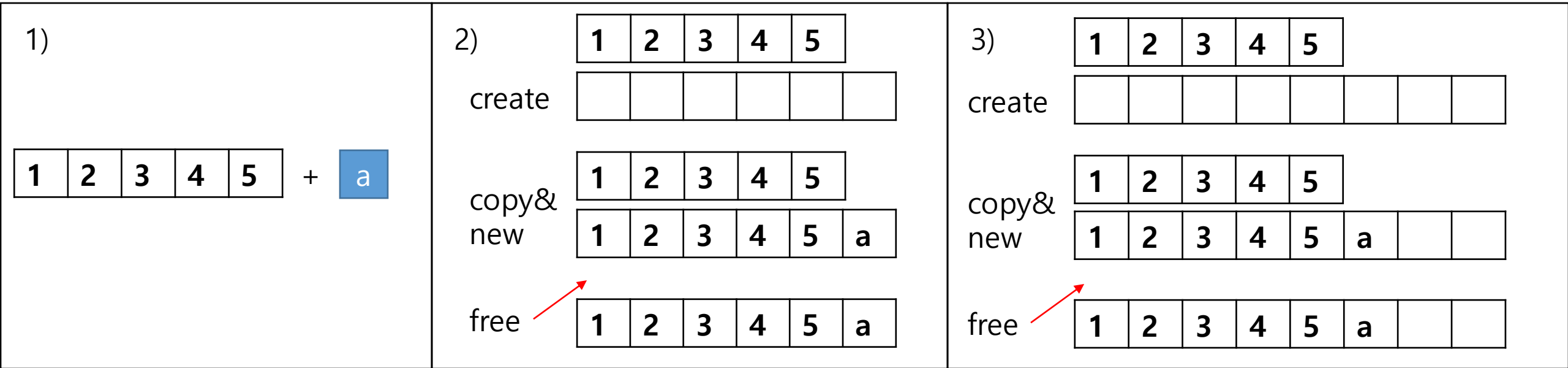
```
arr = list(range(20))
arr[1] = arr[0]+100
print(arr)
```

Q. What is the time complexity of this operation?

O(1)
create, copy, input new one, delete old one

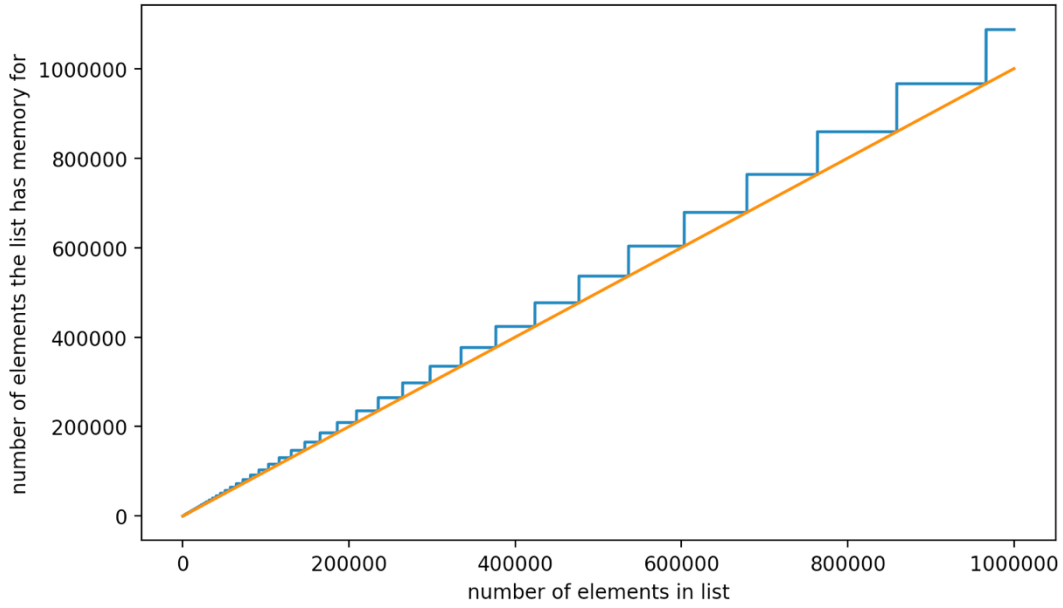
- Resize

```
arr.append('a')
>> before [0, 100, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
>> after [0, 100, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 'a']
```



Lists

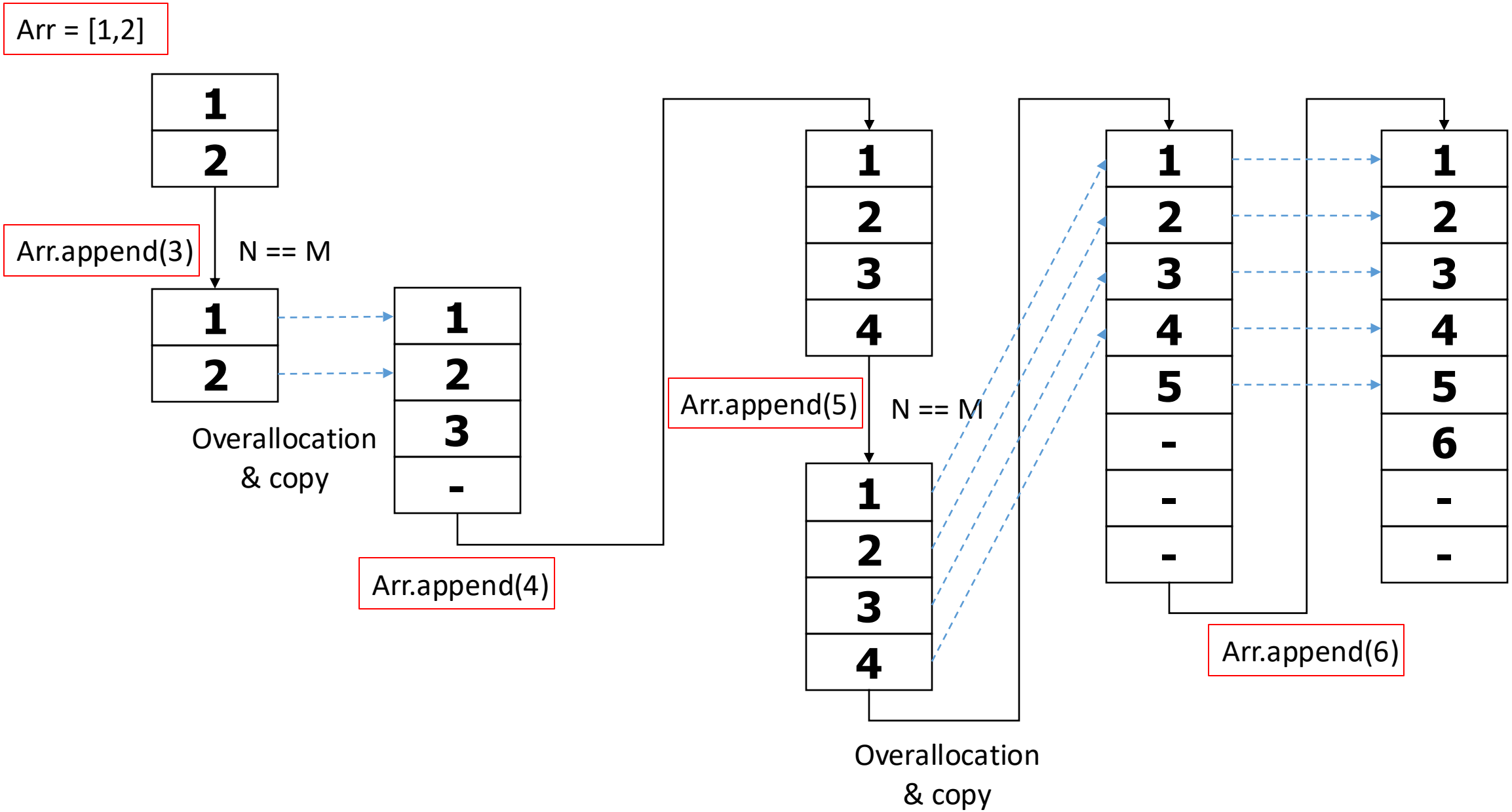
- When appending data, python utilizes the extra memory and increase the effective size of the list, *N*. Effective size (유효크기)
- When $N == M$ (current allocated memory), python create new list with more extra memory.



- 1개만 할당하지 않고, 여러 개를 할당하는 이유는 메모리 할당 operation이 무겁고 느리기 때문에,
- 리스트를 한번 추가하면 그 뒤로도 추가할 확률이 높다고 판단하고 요구한 크기보다 더 많은 메모리를 할당함.
  → 메모리할당과 복사요청 횟수를 줄이기 위함.

https://github.com/python/cpython/blob/ffd9753/Objects/listobject.c#L52-L65

Illustration of how a list is mutated according to 'appends'



Comparison between list comprehension and append op.

```
!pip install memory_profiler
import memory_profiler
%load_ext memory_profiler

%memit [i for i in range(100_000)]

>> peak memory: 193.45 MiB, increment: 2.13 MiB


%memit arr2 = []
for i in range(100_000):
    arr2.append(i)


>> peak memory: 201.89 MiB, increment: 10.29 MiB
```

x5

10만개의 데이터 할당에 대한 2가지 다른 예제
리스트 내포(list comprehension)으로 만들 때보다 append를 사용하면 5배 더 많이 할당하는 것을 관찰할 수 있음.
컴퓨터 마다 다를 수 있고 파이썬 버전 마다 다를 수 있다.

Tuples

Tuples

- Impossible to Data update

```
tu = tuple(range(5))
print(tu)

>> (0, 1, 2, 3, 4)


tu[0] = 5

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-17-8575601f00af> in <module>
      1 tu = tuple(range(5))
      2 print(tu)
----> 3 tu[0] = 5

TypeError: 'tuple' object does not support item assignment
```

Tuples

- Tuple do not support resizing, but it is possible to concatenate two tuples into new one.

```
tu0 = (1,2,3)
tu1 = (4,5,6)
tu0 + tu1

>> (0, 1, 2, 3, 4, 5, 6)
```

- Do not allocate any extra memory for new tuple.

  >> Tuple allocate memory and copy every time something is added !

  Time complexity of iteration based copy is O(n)

  >> If data guarantee static, tuple is more lightweight than list.

  *In the case of `memcpy`, ideally its time complexity is O(1).

  List는 append를 사용하지 않아서 여유공간을 할당 되어있지 않더라도, Tuple보다 메모리 더 잡아 먹음.
  List를 관리하기 위한 다양한 상태정보를 관리가 필요함 (메모리 입장에서 이 Overhead는 크지 않음.)
  (But, List가 extremely large 하다면?)

Tuples

- <u>Resource cashing</u>

- GC frees unused memory for other application / variables, however small sized tuples (1-20 size) do not return to the system even they are not used; up to 20,000 of each size for future use.

  >> We don't need communication with OS to memory allocation.

예) 2-tuple: 20,000
3-tuple: 20,000
10-tuple: 20,000

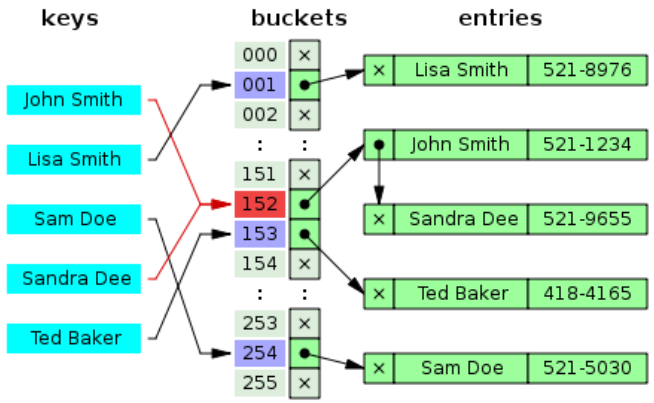
```
%timeit arr_list = [0,1,2,3,4,5,6,7,8,9]
%timeit arr_tuple = (0,1,2,3,4,5,6,7,8,9)
```

```
>> 123 ns ± 1.13 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)
>> 22.9 ns ± 0.489 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)
```

Dictionary

Dictionaries and sets

- Dictionaries and sets are Hash Table.
  - Lookup time on Lists and Tuple O(log n), at best, if we don't know intrinsic order.
  - Dictionaries and Sets give us O(1) lookup time at best.
  - However, need more memory and the actual speed depends on the *hash code* and *compression function*.



Dictionaries and sets

- Dictionaries and sets are Hash Table.

<Dictionary>

```
phonebook = {
  "Inwook Shim": "032-123-4567",
  "Jihong Min": "042-777-7777"
}
print(phonebook["Inwook Shim"])
```

O(1)

<List>

```
def search_person(phonebook, name):
  for k, v in phonebook:
    if k == name:
      return v
  return None

phonebook = [
    ("Inwook Shim", "032-123-4567"),
    ("Jihong Min", "042-777-7777")
]
print(search_person(phonebook, "Inwook Shim"))
```

O(n)

Dictionaries and sets

- Inserting: key → hash_code → hashed value → compression → index
- Probing
- Load factor
- Resizing

Hash Functions and Entropy

- Objects in Python are generally heshable (해쉬가 가능하다.)
- Built-in `__hash__` and `__cmp__`
  - The default `__hash__` function returns the object's placement in memory using the built-in `__id__ function`
  - The default `__cmp__` compares the numerical value of the object's placement in memory.

```
class Point(object):
  def __init__(self, x, y):
    self.x, self.y = x, y

p1 = Point(1,1)
p2 = Point(1,1)
pset = set([p1, p2])
print(pset)
Point(1,1) in pset
```

해시테이블 내에서 충돌이 발생하지 않음 (메모리)

메모리 기반 해쉬테이블 구성이 아니라
실제 내용을 기반으로 하는 사용자 정의 해시함수를 작성할 수 있음

```
>>> {<__main__.Point object at 0x7f436ccc8710>, <__main__.Point object at 0x7f436ccc8490>}
>>> False
```

Hash Functions and Entropy

- Custom hash function to compare actual contents of the object, not memory

```
class Point(object):
  def __init__(self, x, y):
    self.x, self.y = x, y

  def __hash__(self):
    return hash((self.x, self.y))

  def __eq__(self, other):
    return self.x == other.x and self.y == other.y

p1 = Point(1,1)
p2 = Point(1,1)
pset = set([p1, p2])
print(pset)
Point(1,1) in pset
```

a custom-selected hash function should be careful to evenly distribute hash values in order to avoid collisions.

If most keys have collision, try probing.

사용자 정의 해시 함수에서 충돌을 피하려면 해시 값이 균일하게 분포되도록 신경 써야한다.

```
>>> {<__main__.Point object at 0x7f436cd34490>}
>>> True
```

Hash Functions and Entropy

- Entropy : How well distributed a hash function

$$S = -\Sigma_i \, p(i) \ \cdot \ \log(p(i))$$

- $p(i)$ is the probability that the hash function gives hash i.
- The entropy is maximized when every hash value has equal probability.
  - Called as ideal hash function

$P(i)$해시 함수가 i를 출력할 확률
모든 확률 값이 같을 때, 엔트로피 최대, 이때가 이상적인 해쉬펑션

Example of Hash Table

```python
import string
import timeit

class BadHash(str):
  def __hash__(self):
    return 42
class GoodHash(str):
  def __hash__(self):
    """This is a slightly optimized version of twoletter_hash """
    return ord(self[1]) + 26 * ord(self[0]) - 2619

baddict = set()
gooddict = set()
for i in string.ascii_lowercase:
  for j in string.ascii_lowercase:
    key=i+j
    baddict.add(BadHash(key))
    gooddict.add(GoodHash(key))

badtime = timeit.repeat(
  "key in baddict",
  setup = "from __main__ import baddict, BadHash; key = BadHash('zz')",
  repeat = 3,
  number = 1_000_000,
)
goodtime = timeit.repeat(
  "key in gooddict",
  setup = "from __main__ import gooddict, GoodHash; key = GoodHash('zz')",
  repeat = 3,
  number = 1_000_000,
)
print(f"Min lookup time for baddict: {min(badtime)}")
print(f"Min lookup time for gooddict: {min(goodtime)}")
```

```
>>> Min lookup time for baddict: 19.521510530999876
>>> Min lookup time for gooddict: 0.39169318600033876
```

Namespace

Namespace

- Whenever a variable, function, or module is invoked in Python, there is a hierarchy that determines where it looks for these objects.

- Find local() array : search entries for all local variables. If a target object is not exist in local() array space, then (next line)
  - Find global() dictionary : search the object in global() dictionaries. (Not Exist?)
    - Find __builtin__ object is searched.
- These calling tree effects to computing performance.

Example of Namespace

```
def outer_func():
  a = 20
  def inner_func():
    a = 30
    print(locals())
  inner_func()
  print(locals())

a = 10
outer_func()
print(locals())
```

```
>>> a = 30
>>> a = 20
>>> a = 10
```

https://www.programiz.com/python-programming/namespace

Example of Namespace

```python
def outer_func():
  a = 20
  def inner_func():
    a = 30
    print(locals())
  inner_func()
  print(locals())

a = 10
outer_func()
print(locals())
```

```
>>> {'a': 30}
>>> {'a': 20, 'inner_func': <function outer_func.<locals>.inner_func at 0x7fb3cfe1b200>}
>>> {'__name__': '__main__', '__doc__': 'Automatically created module for IPython interactive environment', '__package__': None, '__loader__': None, '__spec__': None, '__builtin__': <module 'builtins' (built-in)>, '__builtins__': <module 'builtins' (built-in)>, '_ih': ['', 'def outer_func():\n a = 20\n def inner_func():\n a = 30\n \n inner_func()\n\na = 10\nouter_func()\nprint("Namespace:", __name__)', 'def outer_func():\n a = 20\n def inner_func():\n a = 30\n print(locals())\n \n inner_func()\n print(locals())\n\na = 10\nouter_func()\nprint(locals())'], '_oh': {}, '_dh': ['/content'], 'In': ['', 'def outer_func():\n a = 20\n def inner_func():\n a = 30\n \n inner_func()\n\na = 10\nouter_func()\nprint("Namespace:", __name__)', 'def outer_func():\n a = 20\n def inner_func():\n a = 30\n print(locals())\n \n inner_func()\n print(locals())\n\na = 10\nouter_func()\nprint(locals())'], 'Out': {}, 'get_ipython': <bound method InteractiveShell.get_ipython of <google.colab._shell.Shell object at 0x7fb3ddbee390>>, 'exit': <IPython.core.autocall.ZMQExitAutocall object at 0x7fb3da386450>, 'quit': <IPython.core.autocall.ZMQExitAutocall object at 0x7fb3da386450>, '_': '', '__': '', '___': '', '_i': 'def outer_func():\n a = 20\n def inner_func():\n a = 30\n \n inner_func()\n\na = 10\nouter_func()\nprint("Namespace:", __name__)', '_ii': '', '_iii': '', '_i1': 'def outer_func():\n a = 20\n def inner_func():\n a = 30\n \n inner_func()\n\na = 10\nouter_func()\nprint("Namespace:", __name__)', 'outer_func': <function outer_func at 0x7fb3cfe1b320>, 'a': 10, '_i2': 'def outer_func():\n a = 20\n def inner_func():\n a = 30\n print(locals())\n \n inner_func()\n print(locals())\n\na = 10\nouter_func()\nprint(locals())'}
```

Example of Namespace (DIY)

```
import math
from math import sin
```

?

```
def builtin_sin(x):
  res = 1
  for _ in range(1000):
    res += math.sin(x)
  return res
```

161 µs ± 1.7 µs per loop

```
def global_sin(x):
  res = 1
  for _ in range(1000):
    res += sin(x)
  return res
```

133 µs ± 3.09 µs per loop

18%

```
def local_sin(x, sin=math.sin):
  res = 1
  for _ in range(1000):
    res += sin(x)
  return res
```

118 µs ± 832 ns per loop

27%

Example of Namespace

```python
import math
from math import sin

def builtin_sin(x):
  res = 1
  for _ in range(1000):
    res += math.sin(x)
  return res

def global_sin(x):
  res = 1
  for _ in range(1000):
    res += sin(x)
  return res

def local_sin(x, sin=math.sin):
  res = 1
  for _ in range(1000):
    res += sin(x)
  return res
```

```
8           18 LOAD_FAST                1 (res)
            20 LOAD_GLOBAL              1 (math)
            22 LOAD_METHOD              2 (sin)
            24 LOAD_FAST                0 (x)
            26 CALL_METHOD              1
            28 INPLACE_ADD
            30 STORE_FAST               1 (res)
            32 JUMP_ABSOLUTE           14
       >>   34 POP_BLOCK
```

```
14          18 LOAD_FAST                1 (res)
            20 LOAD_GLOBAL              1 (sin)
            22 LOAD_FAST                0 (x)
            24 CALL_FUNCTION            1
            26 INPLACE_ADD
            28 STORE_FAST               1 (res)
            30 JUMP_ABSOLUTE           14
       >>   32 POP_BLOCK
```

```
20          18 LOAD_FAST                2 (res)
            20 LOAD_FAST                1 (sin)
            22 LOAD_FAST                0 (x)
            24 CALL_FUNCTION            1
            26 INPLACE_ADD
            28 STORE_FAST               2 (res)
            30 JUMP_ABSOLUTE           14
       >>   32 POP_BLOCK
```

Conclusion

- List and Tuple are fast and low-overhead data structure (object) when data already has intrinsic ordering.
  - Otherwise, we need sorting and searching.
- We must consider to properly understanding how much overallocation will be happened, when using Lists.
- Tuples create quickly and lower overhead than List, but the value cannot be updated.

- Dictionaries and Sets provide a good way to store data that can be accessed by 'key.'
- Hash function affects the performance of data searching.
- Understanding how dictionaries work is important!

  >> this guides how to organize your data and code when you use Python (Dictionaries are utilized in Python's internal functionality.)

Iterator and Generator

loop in Python

- Loop in other language (e.g., C)

```
for (i = 0; i < N; i++) {
    do_something;
}
```

- Loop in Python

```
for i in range(N):
  do_something(i)
```

- Does `range` function generates all of the data we need for loop?
- How about the following codes?

```
N = 100_000_000
for i in range(N):
  do_something(i)
```

Does Python create 100_000_000 sized array?

Generator functions do lazily evaluation (lazy iterator)
- Keep code readability
- Memory efficient

Generator를 사용하면 이런 종류의 함수를 필요할 때마다 지연계산을 할 수 있어서 성능상에 영향을 주지 않을 수 있다.
Lazy evaluation 미리 만들어내는 것이 아니라, 이 함수가 필요할 때마다 계산
1,2,3, 증가하는게 아니라 특정 패턴의 index 작업이 필요한 경우에 유용

Comparison list and generator

- Loop using List

```
def fibonacci_list(num_items):
  numbers = []
  a, b = 0, 1
  while len(numbers) < num_items:
    numbers.append(a)
    a, b = b, a + b
  return numbers
```

- Generator

```
def fibonacci_gen(num_items):
  a, b = 0, 1
  while num_items:
    yield a
    a, b = b, a + b
    num_items -= 1
```

Question: yield and generator?

Generator : lazy iterator 필요할 때 생산해서 반환

```
%timeit fibonacci_list(100_000)
%memit fibonacci_list(100_000)
```

```
>>> 971 ms ± 674 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
>>> peak memory: 622.71 MiB, increment: 0.81 MiB
```

```
%timeit fibonacci_gen(100_000)
%memit fibonacci_gen(100_000)
```

```
>>> 243 ns ± 5.68 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
>>> peak memory: 622.02 MiB, increment: 0.01 MiB
```

Python Grammer, yield

- return

```
def return_num():
  return list(1,2,3)

print(return_num())
>>> [1, 2, 3]
```

- yield

```
def yield_num():
  yield 1
  yield 2
  yield 3

print(yield_num())
<generator object yield_num at 0x7f33e19b28d0>
```

- Boundless data generation

```
def yield_num():
  while True:
    yield 1
    yield 2
    yield 3

for num in yield_num():
  print(num)
```

- Please check, “generator comprehension”, “yield from”

Comparison list and generator

- Loop using List

```
def fibonacci_list(num_items):
  numbers = []
  a, b = 0, 1
  while len(numbers) < num_items:
    numbers.append(a)
    a, b = b, a + b
  return numbers
```

1. `fibonacci_list(1000)` do append to the `numbers` list
2. return the `numbers` list

*memory overhead

- Generator

```
def fibonacci_gen(num_items):
  a, b = 0, 1
  while num_items:
    yield a
    a, b = b, a + b
    num_items -= 1
```

1. `fibonacci_gen(1000)` return values according to request.
2. every time the function gets to the yield, this function feeds a value
3. When another value is requested, the function resume with maintaining its previous state.

*nomemory overhead

- As a result, both functions do almost same number of calculation, but fibo_list consumes 1,000 times or more.

전체적으로 보았을 때, generator가 유리해 보인다. 다만, 내가 구현하려는 목적에 따라서 다르게 평가될 필요가 있다.
예, 전체적인 성능에 손해를 보더라도 iteration 그 자체의 response 가 빠르게 요구되는 경우.

Python intrinsic generators

- map
- zip
- filter
- Reversed
- enumerate

- 모든 경우에 만능이 아니니 반드시 목적에 맞게 사용하는 것이 중요-